

13 *Sequence-to-Sequence and Attention*

<i>Objective</i>	1
<i>Theory and Examples</i>	2
<i>Basic Seq2Seq Model</i>	2
<i>Attention</i>	4
<i>Self-Attention</i>	11
<i>Input Embedding</i>	17
<i>Positional Encoding</i>	20
<i>Full Decoder-Only Transformer</i>	23
<i>Epilogue</i>	25
<i>Further Reading</i>	26
<i>Summary of Results</i>	27
<i>Solved Problems</i>	32
<i>Exercises</i>	40

Objective

Seq2Seq is a family of models: one sequence (e.g., an English sentence) is input to the model, which then produces an output sequence (e.g., a French sentence). The model consists of an encoder that processes the input sequence and produces some context that is passed to the decoder, which then generates the output sequence. To improve the operation of Seq2Seq models, attention was introduced, and then, eventually, the entire Seq2Seq model was implemented using only attention – the transformer model.

Seq2Seq and Attention

Theory and Examples

Sequence-to-sequence (Seq2Seq) models were originally developed for machine translation. An input sequence is a sentence in one language, and the output sequence is a sentence in a second language. The Seq2Seq structure has two parts:

- The *Encoder* processes the input sequence and produces *context*.
- The *Decoder* generates the output sequence from the context.

Many encoder and decoder structures have been used. The original paper [Sutskever et al., 2014] used long short term memory (LSTM) networks for both encoder and decoder. In follow-up work, attention connections were added between encoder and decoder [Bahdanau et al., 2014], and later recurrent connections were eliminated, producing the attention-only transformer network [Vaswani et al., 2017]. There are now countless variations of the Seq2Seq model. Our focus here is to introduce the Seq2Seq model in its basic form, and then to show the progression as attention is added and then feedback connections are removed. This can be demonstrated using the simplest Seq2Seq models, in which a standard recurrent neural network (RNN) is used for both encoder and decoder.

The basic Seq2Seq model is introduced in the next section.

Basic Seq2Seq Model

The basic recurrent network, which we introduced in the previous chapter, is shown in Figure 13.1. Equations 13.1 and 13.2 define the network operation.

$$\mathbf{a}^1(t) = \text{tansig} \left(\mathbf{IW}^{1,1} \mathbf{p}(t) + \mathbf{LW}^{1,1} \mathbf{a}^1(t-1) + \mathbf{b}^1 \right) \quad (13.1)$$

$$\mathbf{a}^2(t) = \mathbf{LW}^{2,1} \mathbf{a}^1(t) + \mathbf{b}^2 \quad (13.2)$$

We will use this network as both the encoder and the decoder of the Seq2Seq model, which is shown in Figure 13.2. The first RNN is the *Encoder*, and the second RNN is the *decoder*. The $\mathbf{u}(t)$ input sequence might represent English words and the output sequence $\mathbf{y}(t)$ might represent French words.

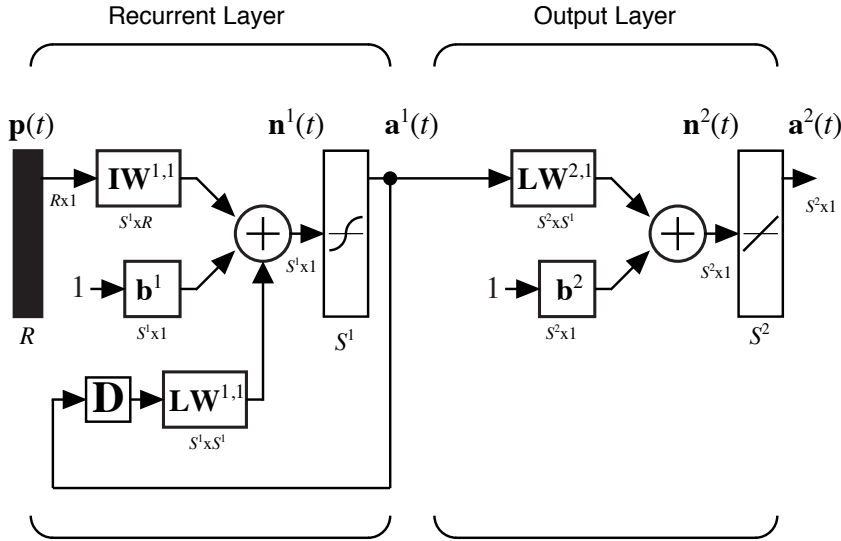
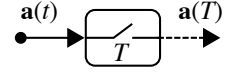


Figure 13.1: Basic Recurrent Network

The encoder takes the input sequence, which has T time points (words, or word fragments), and produces a layer output $\mathbf{a}^1(t)$. This output is sampled at the final time point $t = T$. This is represented by the sampler block, shown in the right margin. The dotted line at the output of the sampler indicates that this value is fixed, and does not change with time. The operation of the encoder is defined by Eq. 13.3.



$$\mathbf{a}^1(t) = \text{tansig} \left(\mathbf{IW}^{1,1} \mathbf{p}(t) + \mathbf{LW}^{1,1} \mathbf{a}^1(t-1) + \mathbf{b}^1 \right) \quad (13.3)$$

The decoder initializes its state with the final state of the encoder. The final state of the encoder also becomes a constant input to the first layer of the decoder. (There are many different formulations of the seq2seq model. In some cases the final encoder state is only used to initialize the state of the decoder.) This final encoder state is sometimes referred to as the *context*. The context is a summary of the input sequence. The operation of the decoder is defined by Equations 13.4 and 13.5.

$$\mathbf{a}^2(t) = \text{tansig} \left(\mathbf{LW}^{2,3} \mathbf{a}^3(t-1) + \mathbf{LW}^{2,1} \mathbf{a}^1(T) + \mathbf{LW}^{2,2} \mathbf{a}^2(t-1) + \mathbf{b}^2 \right) \quad (13.4)$$

$$\mathbf{a}^3(t) = \mathbf{LW}^{3,2} \mathbf{a}^2 + \mathbf{b}^3 \quad (13.5)$$

Seq2Seq and Attention

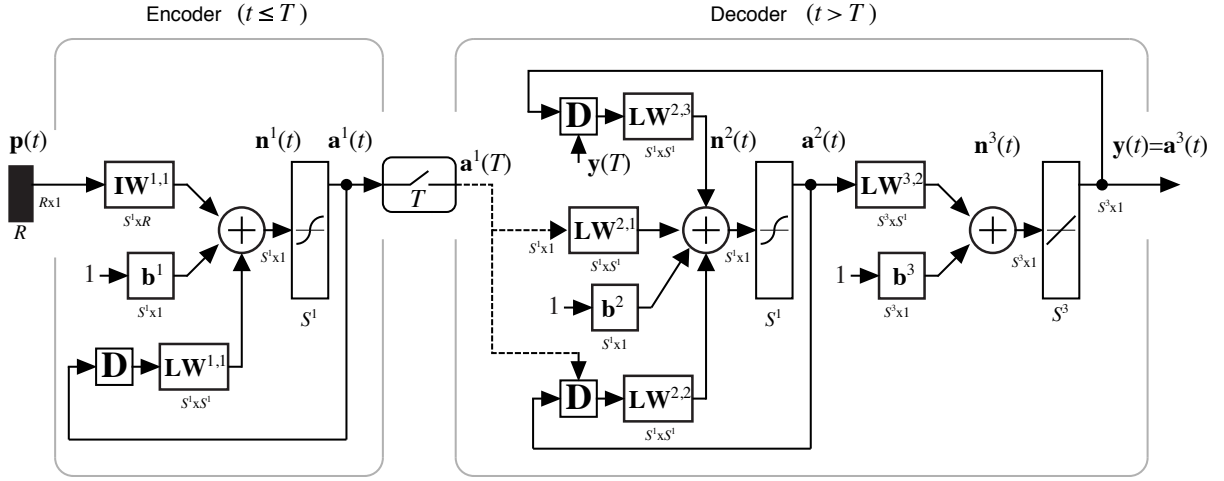


Figure 13.2: Sequence to Sequence Network

Attention

The recurrent layer of the encoder is able to store information from the beginning of the input sequence (subsequently provided as context for the decoder), but it may not be able to store it in the most efficient way. (We know from Chapter 11 that the impulse response of a stable linear recurrent network decays exponentially.) To provide more flexibility in creating the context, we can add a tapped delay line of previous values of the encoder state, rather than using just the final value. This process is called *attention* (see [Bahdanau et al., 2014]). (The concept is similar to the general IIR network of Figure 11.13, in which there is a TDL of the input sequence in addition to the recurrent connections.)

The first step in implementing attention is to store previous encoder states in a horizontally-stacked TDL (hstack), as shown in Figure 13.3, in which the vectors are stacked as columns. There is one $S \times 1$ vector coming in to the TDL_hstack, and a $S \times D$ matrix going out, where $D - 1$ is the number of delays in the TDL. In other words,

$$\mathbf{A}(t) = \begin{bmatrix} \mathbf{a}(t) & \mathbf{a}(t-1) & \dots & \mathbf{a}(t-D+1) \end{bmatrix} \quad (13.6)$$

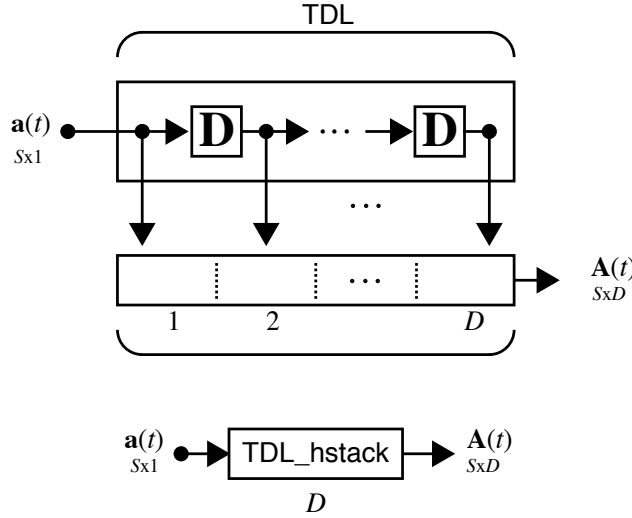


Figure 13.3: Horizontal Stacked Tapped Delay Line

We want to combine these vectors together in some way to form the context vector to send to the decoder (in place of just the final encoder state). The question is how much should each previous encoder state contribute to the context. Also, we may want the context to change at each time step of the decoder operation. For example, when translating a given English sentence to French, as we get to a given French word in the translation, it may depend on different combinations of English words in the original sentence. One approach could be to find out how correlated the previous decoder state is to each previous encoder state. This could be done by finding the inner product between the previous decoder state and all previous encoder states, as in the following equation.

$$\mathbf{n}^4(t) = \left[\mathbf{A}^1(T) \right]^T \mathbf{a}^2(t-1) \quad (13.7)$$

We should then normalize these correlations by using the softmax activation function, as in

$$\mathbf{a}^4(t) = \text{softmax} \left(\mathbf{n}^4(t) \right) \quad (13.8)$$

Finally, we combine the previous encoder state vectors according to the relative amount of correlation with the previous decoder state.

$$\mathbf{a}^5(t) = \mathbf{A}^1(T) \mathbf{a}^4(t) \quad (13.9)$$

Seq2Seq and Attention

The resulting $\mathbf{a}^5(t)$ is the context vector that will be passed to the decoder. A diagram of this *dot product attention* is shown in Figure 13.4, where the T inside the circle is the transpose operator (see margin).

$$\mathbf{A} \rightarrow \mathbf{A}^T$$

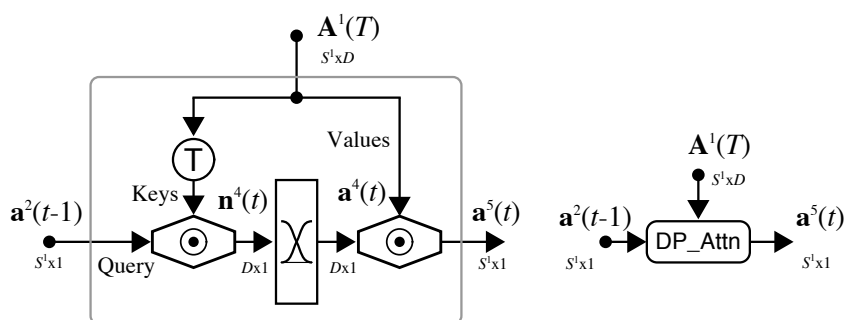


Figure 13.4: Dot Product Attention

The idea of attention is sometimes generalized as operations between a *query* vector and sets of *key* and *value* pairs. The query is compared to each of the keys to determine how much of each corresponding value to include in the result. In our case, the query is the previous decoder state $\mathbf{a}^2(t-1)$, the keys are all previous encoder states, and the values are also the previous encoder states.

Background on Query, Key and Value

The terms Query, Key and Value come from database terminology. For example, if we have a database that contains the names and ages of employees, we might want to find someone's age by providing a Query to the database, which could be the employee's last name. In the table below we provide the Query *Floyd* to the database. The Query is compared to all of the Keys (last name) in the database. The closest Key is *Floyd*. The corresponding Value of *24* is returned.

Rank	Last Name (Key)	Age (Value)
2nd	Flowers	35
1st	Floyd	24
3rd	Fly	57
...	...	...

In attention, the values of different entries are combined in proportion to how close the query is to each of the keys. To illustrate, consider the query and keys/values given below.

$$\frac{2}{+2} = 4$$

$$\mathbf{A}^1(T) = \begin{bmatrix} 0.7 & -0.6 & 0.2 & -0.8 \\ 0.3 & 0.5 & -0.7 & -0.5 \\ -0.6 & 0.1 & 0.6 & 0.4 \end{bmatrix} \text{ Keys/Values}$$

$$\mathbf{a}^2(t-1) = \begin{bmatrix} 0.4 \\ 0.1 \\ -0.3 \end{bmatrix} \text{ Query}$$

Now perform the attention inner product operation between the query and keys.

$$\mathbf{n}^4(t) = \left[\mathbf{A}^1(T) \right]^T \mathbf{a}^2(t-1) = \begin{bmatrix} 0.49 \\ -0.22 \\ -0.17 \\ -0.49 \end{bmatrix}$$

Passing these correlations through the softmax activation function produces the attention weights $\mathbf{a}^4(t)$.

$$\mathbf{a}^4(t) = \text{softmax} \left(\mathbf{n}^4(t) \right) = \begin{bmatrix} 0.420 \\ 0.206 \\ 0.217 \\ 0.157 \end{bmatrix} \text{ Attention Weights}$$

Now we use the attention weights to combine the values.

Seq2Seq and Attention

$$\begin{aligned}
 \mathbf{a}^5(t) &= \mathbf{A}^1(T) \mathbf{a}^4(t) \\
 &= 0.420 \begin{bmatrix} 0.7 \\ 0.3 \\ -0.6 \end{bmatrix} + 0.206 \begin{bmatrix} -0.6 \\ 0.5 \\ 0.1 \end{bmatrix} + 0.217 \begin{bmatrix} 0.2 \\ -0.7 \\ 0.6 \end{bmatrix} + 0.157 \begin{bmatrix} -0.8 \\ -0.5 \\ 0.4 \end{bmatrix} \\
 &= \begin{bmatrix} 0.087 \\ -0.002 \\ -0.038 \end{bmatrix}
 \end{aligned}$$

Note that the query has the highest correlation with the first key, therefore the first value contributes the most to the result of the attention operation, $\mathbf{a}^5(t)$.

Figure 13.5 shows the seq2seq network with dot product attention added. The difference between the standard seq2seq model and the one with attention is that $\mathbf{a}^5(t)$ is the context vector that is input to Layer 2, instead of $\mathbf{a}^1(T)$. The $\mathbf{a}^5(t)$ context will change with each time step, whereas $\mathbf{a}^1(T)$ is fixed. The $\mathbf{a}^4(t)$ vector (whose elements are sometimes referred to as the attention weights) will determine how much each $\mathbf{a}^1(t)$ will contribute to the context at each step of the decoder. If the first element of $\mathbf{a}^4(t)$ is equal 1 (and the other elements are consequently equal to 0), then the context will again be $\mathbf{a}^1(T)$, as in the standard seq2seq model. Otherwise, the context will include additional time points of the encoder state. This means that the decoder will be paying *attention* to different encoder states at different steps of the decoder operation.

The inner product between the query vector $\mathbf{a}^2(t-1)$ (for $t > T$) and the key vectors $\mathbf{a}^1(t)$ (for $t \leq T$) is a measure of the similarity between the query and each of the keys. This use of inner product for measuring similarity is discussed in Chapter 5 of [NND2](#). As that chapter indicated, there are many forms of inner product.

If we have two vectors, $\mathbf{x}$ and $\mathbf{y}$, then the standard inner product is the dot product $\mathbf{x}^T \mathbf{y}$. Another common inner product is the weighted dot product $\mathbf{x}^T \mathbf{W} \mathbf{y}$, where the symmetric, positive definite weighting matrix $\mathbf{W}$ is used to emphasize certain components of the dot product. This concept can be easily incorporated into the attention mechanism. We just need to replace Eq. 13.7 with

$$\mathbf{n}^4(t) = \left[\mathbf{A}^1(T) \right]^T \mathbf{W}_a \mathbf{a}^2(t-1) \quad (13.10)$$

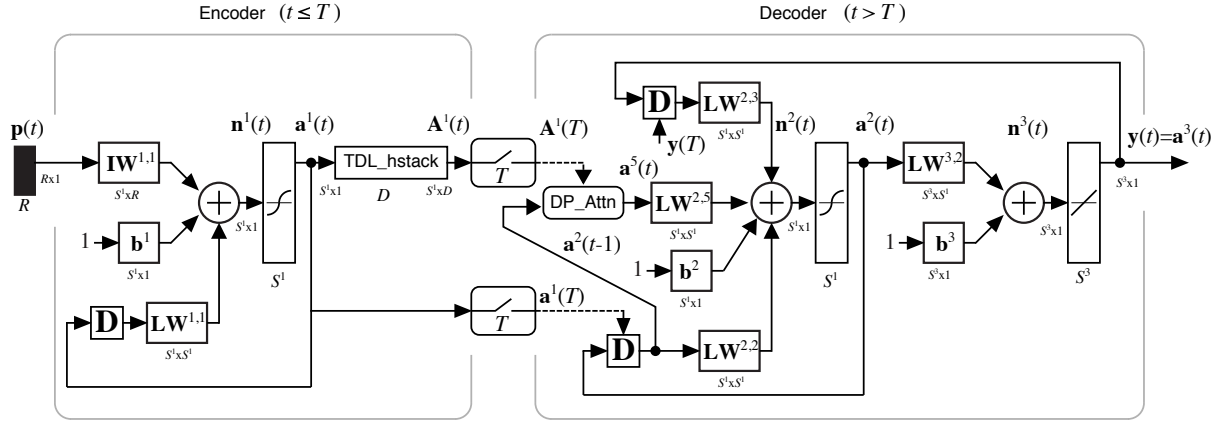


Figure 13.5: Sequence to Sequence Network With Attention

where W_a is the *attention weighting matrix*. This type of attention has been referred to as general attention, although it might be more precise to call it *general dot product attention* or weighted dot product attention. Figure 13.6 is a representation of this general dot product attention.

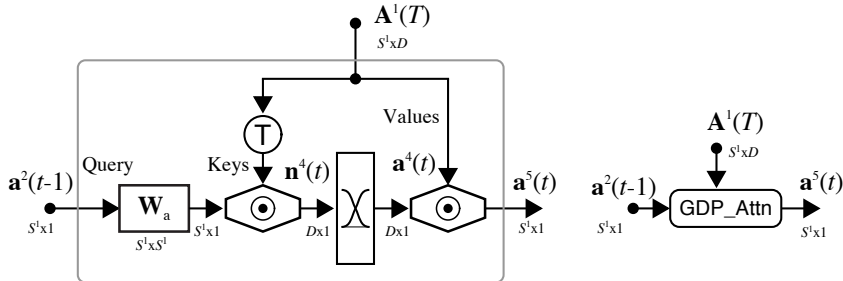


Figure 13.6: General Dot Product Attention

The idea of attention is to determine how much the encoder state at each time point (for $t \leq T$) should contribute to the context vector, given the query vector $a^2(t-1)$. Instead of performing an inner product between the query and the keys (encoder states), we could just use a general functional relationship between them. This could be done by stacking (concatenating) the query with each of the keys and passing the result through a multilayer subnetwork, which could be trained as part of the overall seq2seq network.

The first step is to vertically stack, or concatenate, the query $a^2(t-1)$ with the previous encoder states $A^1(T)$.

The vertical stacking operation uses the NumPy broadcasting rules. If one of the arrays has a single column, that column is repeated until the number of columns matches the larger array.

Seq2Seq and Attention

$$\bar{\mathbf{A}}^{1,2}(t) = \begin{bmatrix} & & \mathbf{A}^1(T) \\ \mathbf{a}^2(t-1) & \mathbf{a}^2(t-1) & \dots & \mathbf{a}^2(t-1) \end{bmatrix} \quad (13.11)$$

$$= \begin{bmatrix} \begin{bmatrix} \mathbf{a}^1(T) \\ \mathbf{a}^2(t-1) \end{bmatrix} & \begin{bmatrix} \mathbf{a}^1(T-1) \\ \mathbf{a}^2(t-1) \end{bmatrix} & \dots & \begin{bmatrix} \mathbf{a}^1(T-D+1) \\ \mathbf{a}^2(t-1) \end{bmatrix} \end{bmatrix} \quad (13.12)$$

This matrix is then passed into a two layer network.

$$\mathbf{A}^4(t) = \text{tansig} \left(\mathbf{LW}^{4,2} \bar{\mathbf{A}}^{1,2}(t) + \mathbf{b}^4 \right) \quad (13.13)$$

$$\mathbf{N}^5(t) = \mathbf{LW}^{5,4} \mathbf{A}^4(t) + \mathbf{b}^5 \quad (13.14)$$

$$\mathbf{a}^5(t) = \text{softmax} \left(\left[\mathbf{N}^5(t) \right]^T \right) \quad (13.15)$$

Finally, to get the attention weights, we multiply by the values $\mathbf{A}^1(T)$.

$$\mathbf{a}^6(t) = \mathbf{A}^1(T) \mathbf{a}^5(t) \quad (13.16)$$

Figure 13.7 shows the implementation of *concatenation attention*.

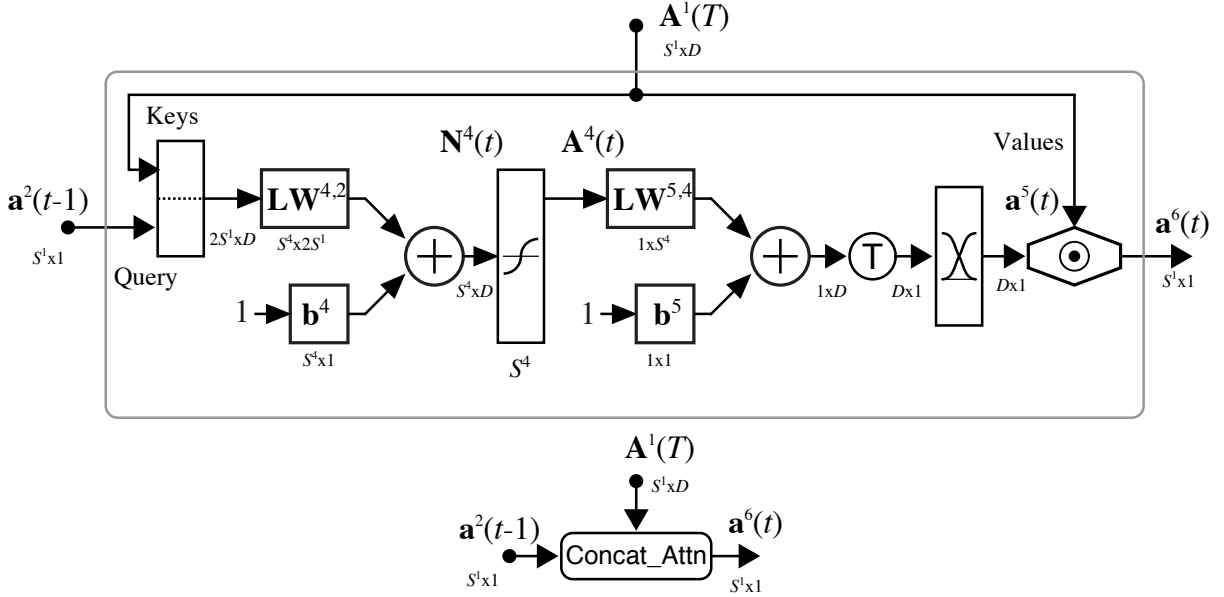


Figure 13.7: Concatenation Attention

Self-Attention

The addition of attention to the Seq2Seq model improved the performance significantly. However, there was a fundamental issue with using recurrent networks in the Seq2Seq model. Recurrent networks need to be implemented sequentially, since the output at time step $t + 1$ cannot be computed until the output at time step t is computed. We could make better use of the parallel operations of GPUs, if we didn't have to perform the operations sequentially. This led to the proposal in [Vaswani et al., 2017] to eliminate the recurrent connections and use only the attention operation.

In the dot product attention operation of Eq. 13.8, we take the dot products between the last decoder state $\mathbf{a}^2(t - 1)$ (the *query*) and the previous encoder states $\mathbf{a}^1(t - d)$ (the *keys* and *values*). Without the recurrent network states, we can only take the dot products between the inputs $\mathbf{p}(t - d)$. Therefore, as in Eq. 13.6, we can create a TDL_hstack of previous input vectors.

$$\mathbf{P}(T) = \begin{bmatrix} \mathbf{p}(T) & \vdots & \mathbf{p}(T - 1) & \vdots & \dots & \vdots & \mathbf{p}(1) \end{bmatrix} \quad (13.17)$$

If we just take dot products among all combinations of inputs, then there are no weights to train to make the dot products more effective. For this reason, the query, key and value vectors will be determined by different linear combinations of the input vector:

$$\mathbf{A}^Q = \mathbf{W}^Q \mathbf{P} \quad (13.18)$$

$$\mathbf{A}^K = \mathbf{W}^K \mathbf{P} \quad (13.19)$$

$$\mathbf{A}^V = \mathbf{W}^V \mathbf{P} \quad (13.20)$$

where $\mathbf{W}^Q$, $\mathbf{W}^K$ and $\mathbf{W}^V$ are trainable matrices.

The attention inner product operation between queries and keys is represented by

$$\mathbf{N}^A = \frac{1}{\sqrt{S^K}} \left[\mathbf{A}^K \right]^T \mathbf{A}^Q \quad (13.21)$$

where the inner products are normalized by dividing by $\sqrt{S^K}$, so that the inner products don't become extremely large and drive the softmax function, which follows, into areas where the gradients will be very small.

Seq2Seq and Attention

The attention weights are determined by passing the output of the inner products through the softmax activation:

$$\mathbf{A}^A = \text{softmax}(\mathbf{N}^A, \text{columns}) \quad (13.22)$$

Note that the softmax operates over the columns of the matrix, so that the sum of the columns of $\mathbf{A}^A$ will be one.

Next, the attention weights multiply the values to produce the output of the attention operation:

$$\mathbf{A}^O = \mathbf{A}^V \mathbf{A}^A \quad (13.23)$$

The total operation is represented in Figure 13.8.

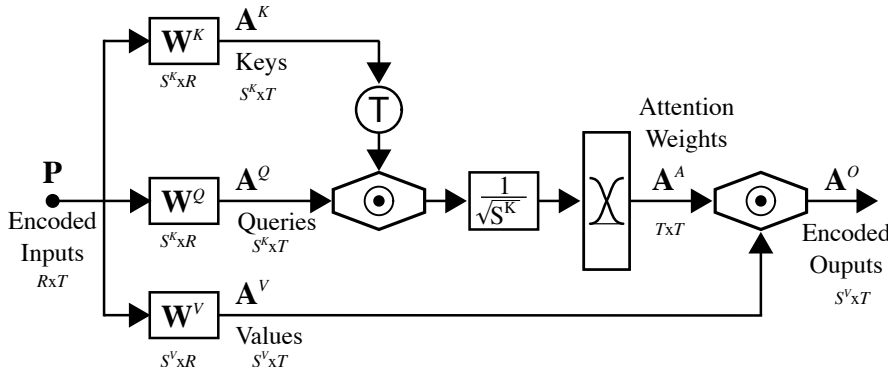


Figure 13.8: Self Dot Product Attention

Because the TDL is *horizontally stacked*, the matrix multiplications operate individually on each time point. This is unlike the focused time delay neural network, shown in Figure 12.1, with the vertical TDL, where the matrix multiplication in the first layer mixes the time points. The operation works like a batch operation, where all the samples in the TDL pass through the network in parallel. **The only place that the samples intermix is in the attention operation.**

This is an important point. In the attention operation, the weights $\mathbf{W}^Q$, $\mathbf{W}^K$ and $\mathbf{W}^V$ work together with the dot product operation to compute the encoded outputs $\mathbf{A}^O$, but it is only the dot product operation that mixes the time points.

It is easier to follow the self-attention operation with an example. Consider the following sequence of inputs:

When the TDL is vertically stacked, the time steps are mixed by matrix multiplication, which mixes rows. When the TDL is horizontally stacked, the different time points appear in different columns, which are not mixed by standard matrix multiplication.

$$\begin{array}{|c|} \hline 2 \\ + 2 \\ \hline 4 \\ \hline \end{array}$$

$$\mathbf{P} = \begin{bmatrix} \mathbf{p}(1) & \mathbf{p}(2) \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 0 \end{bmatrix} \quad (13.24)$$

and assume that the query, key and value weights are given by

$$\mathbf{W}^Q = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \mathbf{W}^K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}, \mathbf{W}^V = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix} \quad (13.25)$$

The query, key and value are then computed from the input sequence as follows:

$$\mathbf{A}^Q = \mathbf{W}^Q \mathbf{P} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} \mathbf{a}^Q(1) & \mathbf{a}^Q(2) \end{bmatrix} \quad (13.26)$$

$$\mathbf{A}^K = \mathbf{W}^K \mathbf{P} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 2 \\ 2 & 0 \end{bmatrix} = \begin{bmatrix} \mathbf{a}^K(1) & \mathbf{a}^K(2) \end{bmatrix} \quad (13.27)$$

$$\mathbf{A}^V = \mathbf{W}^V \mathbf{P} = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 2 & 0 \end{bmatrix} = \begin{bmatrix} \mathbf{a}^V(1) & \mathbf{a}^V(2) \end{bmatrix} \quad (13.28)$$

Now we perform the attention dot product operation.

$$\mathbf{N}^A = \frac{1}{\sqrt{2}} \left[\mathbf{A}^K \right]^T \mathbf{A}^Q = \frac{1}{\sqrt{2}} \begin{bmatrix} 0 & 2 \\ 2 & 0 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 0 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 2 & 0 \\ 2 & 4 \end{bmatrix} \quad (13.29)$$

$$= \begin{bmatrix} \sqrt{2} & 0 \\ \sqrt{2} & 2\sqrt{2} \end{bmatrix} \quad (13.30)$$

$$(13.31)$$

It is helpful to notice specifically how the elements of this matrix are composed of the dot products between the key and query vectors.

$$\mathbf{N}^A = \frac{1}{\sqrt{2}} \begin{bmatrix} \mathbf{a}^K(1)^T \\ \mathbf{a}^K(2)^T \end{bmatrix} \begin{bmatrix} \mathbf{a}^Q(1) & \mathbf{a}^Q(2) \end{bmatrix} \quad (13.32)$$

$$= \frac{1}{\sqrt{2}} \begin{bmatrix} \mathbf{a}^K(1)^T \mathbf{a}^Q(1) & \mathbf{a}^K(1)^T \mathbf{a}^Q(2) \\ \mathbf{a}^K(2)^T \mathbf{a}^Q(1) & \mathbf{a}^K(2)^T \mathbf{a}^Q(2) \end{bmatrix} \quad (13.33)$$

Seq2Seq and Attention

Now we compute the attention weights using the softmax function.

$$\mathbf{A}^A = \text{softmax}(\mathbf{N}^A, \text{columns}) = \begin{bmatrix} 0.5 & 0.06 \\ 0.5 & 0.94 \end{bmatrix} \quad (13.34)$$

The attention weights then multiply the value vectors to get the encoded outputs.

$$\mathbf{A}^O = \mathbf{A}^V \mathbf{A}^A = \begin{bmatrix} 0.5\mathbf{a}^V(1) + 0.5\mathbf{a}^V(2) & 0.06\mathbf{a}^V(1) + 0.94\mathbf{a}^V(2) \end{bmatrix} \quad (13.35)$$

$$= \begin{bmatrix} 1.5 & 1.94 \\ 1 & 0.12 \end{bmatrix} \quad (13.36)$$

Note that each column of the attention weight matrix is used to combine the value vectors. The two elements of the first column of the attention weight matrix are equal, so the first encoded output of the attention operation is an equal combination of the two value vectors. We could say that the first encoded output "attends" equally to the two value vectors. For the second encoded output, the attention mainly focuses on the second value vector (with a weight of 0.94).

*To experiment with the attention response surface, use the Deep Learning Demonstration Attention Response (**dl13atsurf**).*



*To experiment with the response of the attention vectors, use the Deep Learning Demonstration Attention Response (**dl13atvec**).*



MASKED ATTENTION

In the standard attention operation, every encoded output can potentially attend to every encoded input. When the operation is used to predict future values of a sequence (like predicting the next word in a sentence) then we want to restrict the attention to previous inputs only. This can be done with *masked attention*, which ensures that the model can only attend to previous inputs in the sequence, not future ones. For a sequence of length n , the attention scores for positions $j \geq i$ (where i is the current input) are masked

(set to $-\infty$). After the softmax, these positions get a weight of 0, so the model cannot attend to future inputs. A diagram of masked attention is shown in Figure 13.9. See Solved Problem P13.1 for a numerical example of masked attention.

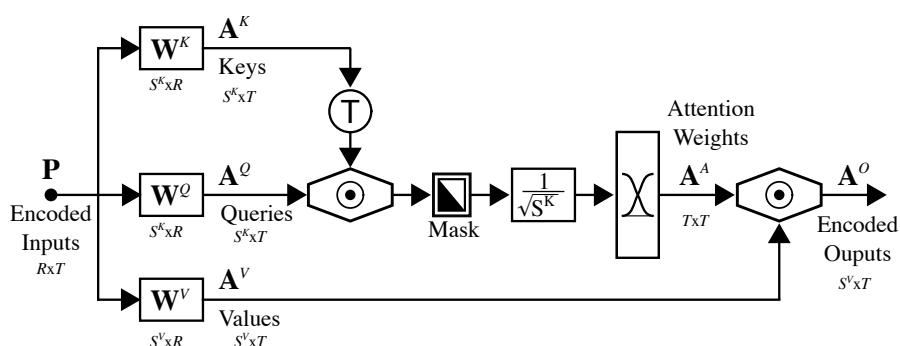


Figure 13.9: Masked Attention

MULTI-HEAD ATTENTION

As we saw in Chapter 8, in the convolution network there can be multiple feature maps in each layer. Each kernel associated with a feature map is extracting a different feature, and simple features can be combined to create more complex features in later layers. In the transformer network there is a similar construct – multiple attention heads. *Multi-head attention* runs several independent attention operations in parallel, each with its own learned weight matrices, then concatenates the results. Figure 13.10 illustrates a two head example.

The standard choice for the dimensions of the heads is to use $S^K = R/H$ and $S^O = R$, where H is the number of heads. The full embedding dimension R is divided among H heads, so each head operates in a lower-dimensional subspace (R/H dimensions). Each head is forced to find a compact, coherent subspace to work in, rather than one large attention operation that may spread its capacity diffusely across R dimensions.

Different heads can learn to capture different types of relationships simultaneously. In a sentence like "The animal didn't cross the street because it was too tired," one head might learn to resolve the pronoun "it" back to "animal" (a coreference relationship), while another head attends to syntactic structure (subject-verb dependencies), and a third might capture positional proximity. No single

Seq2Seq and Attention

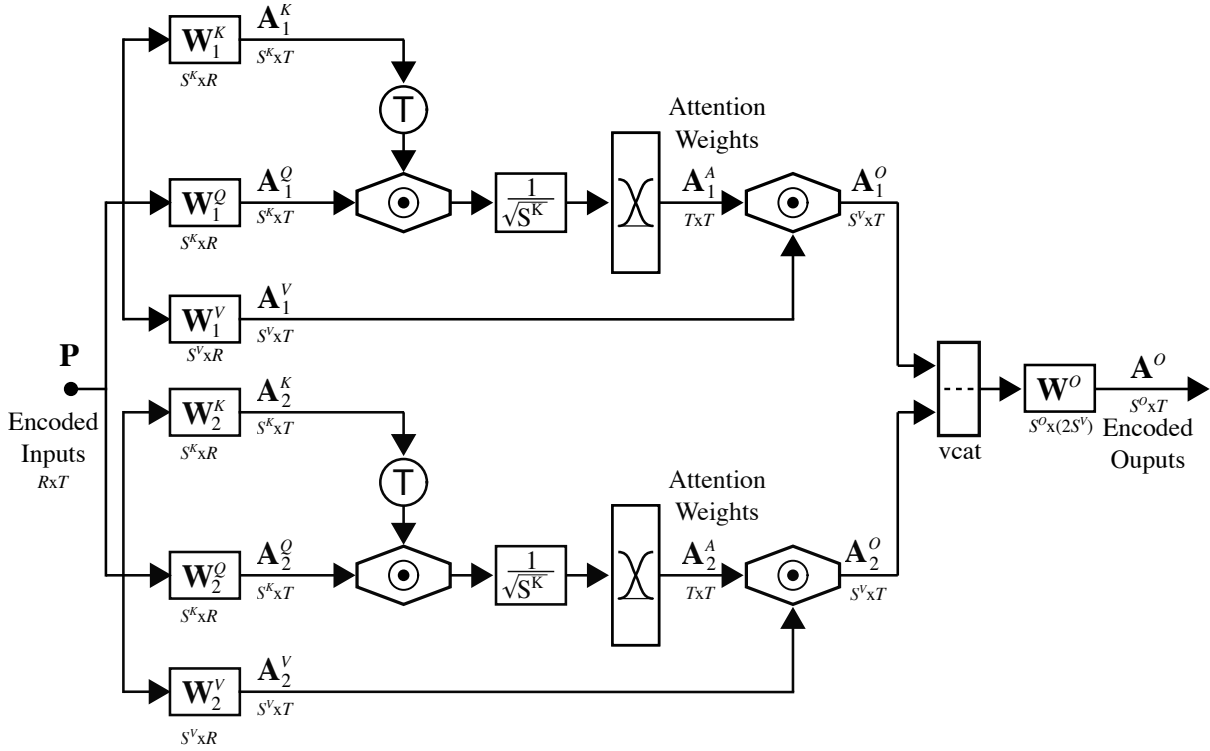


Figure 13.10: Multi-head Attention

attention head could specialize in all of these at once, because the W^Q , W^K , W^V matrices would be pulled in competing directions during training. A numerical example of multi-head attention is given in Solved Problem P13.3.

INDUCTIVE BIAS OF ATTENTION

Self-attention, like convolution, introduces some inductive biases into the network. Some of these biases are useful, while others need to be mitigated, as we will discuss in the next section, to produce the complete *transformer* architecture of [Vaswani et al., 2017]. Some of the inductive biases are:

- **Permutation invariance:** The results of the dot product operation do not depend on where an input appears in the sequence. For example, the input sequences "The dog bit the cat." and "The cat bit the dog." would produce the same dot product calculations. The encoded "cat" token would be the same in both instances.

- **Locality agnosticism:** Unlike the CNN (which has a local receptive field) and the RNN (which operates sequentially), self-attention can directly relate any two elements in the sequence, regardless of their distance. This makes transformers agnostic to locality (limited by the length of its TDL).
- **Dot product similarity:** The dot product measures the cosine similarity (up to a scale factor) between query and key vectors. (See Chapter 5 of [NND2](#).) This biases the model to attend more to elements whose representations are similar in direction.

The permutation invariance can be a problem, because the ordering of items is important. We can use something called *positional encoding* to help the transformer keep track of word order. To make the dot product operation more effective, we can use an *input embedding*. These two concepts are described in the next two sections.

Input Embedding

Since in the attention operation the time steps are only mixed in the dot product operations among the input vectors, it is important that the dot product extracts important information. For example, if the input to the network is text (as in natural language processing), then the dot product between the numerical representations of the words should indicate relationships between the words.

In the self-attention operation, the input matrix $\mathbf{P}$ contains columns $\mathbf{p}(1), \mathbf{p}(2), \dots, \mathbf{p}(T)$, where each $\mathbf{p}(t)$ is a numerical vector representing a word (or token) at time step t . But how do we get from a word like “the” to a numerical vector? This is the role of the *input embedding*. For the attention operation to be successful, the embedding should be developed so that the dot product between embedding vectors indicates how related the corresponding words are.

Suppose we have a small vocabulary of $V = 5$ words:

Index	Word
1	the
2	cat
3	sat
4	on
5	mat

$$\begin{array}{r} 2 \\ +2 \\ \hline 4 \end{array}$$

Seq2Seq and Attention

We want to process the sentence: “the cat sat.” The first step is to represent each word as a numerical vector. The most basic representation is a *one-hot* vector, in which a single element is 1 and the rest are 0. For our vocabulary of size $V = 5$:

$$\mathbf{o}(1) = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \text{ (“the”)}, \quad \mathbf{o}(2) = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \text{ (“cat”)}, \quad \mathbf{o}(3) = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \text{ (“sat”)}$$

where $\mathbf{o}(t)$ is the one-hot vector for the word at time step t . Here $\mathbf{o}(1)$ corresponds to “the” (index 1), $\mathbf{o}(2)$ to “cat” (index 2), and $\mathbf{o}(3)$ to “sat” (index 3).

The one-hot vectors have dimension V (the vocabulary size), which is typically very large (tens of thousands). Also, the one-hot representation does not capture any relationships between words; every word is equally distant from every other word. The *embedding matrix* $\mathbf{W}^E$ (of dimension $R \times V$, where R is the chosen embedding dimension) maps each one-hot vector to a compact, dense vector:

$$\mathbf{p}(t) = \mathbf{W}^E \mathbf{o}(t) \quad (13.37)$$

For this example, let $R = 3$ (so each word is represented by a 3-element vector) and suppose the embedding matrix is:

$$\mathbf{W}^E = \begin{bmatrix} 0.2 & 0.8 & -0.1 & 0.5 & 0.3 \\ -0.5 & 0.3 & 0.7 & -0.2 & 0.6 \\ 0.1 & -0.4 & 0.2 & 0.9 & -0.3 \end{bmatrix} \quad (13.38)$$

(dimension 3×5). Now compute the input vector for each word:

$$\begin{array}{|c|} \hline 2 \\ +2 \\ \hline 4 \\ \hline \end{array}$$

$$\mathbf{p}(1) = \mathbf{W}^E \mathbf{o}(1) = \mathbf{W}^E \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.2 \\ -0.5 \\ 0.1 \end{bmatrix} \text{ ("the")} \quad (13.39)$$

$$\mathbf{p}(2) = \mathbf{W}^E \mathbf{o}(2) = \mathbf{W}^E \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.8 \\ 0.3 \\ -0.4 \end{bmatrix} \text{ ("cat")} \quad (13.40)$$

$$\mathbf{p}(3) = \mathbf{W}^E \mathbf{o}(3) = \mathbf{W}^E \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} -0.1 \\ 0.7 \\ 0.2 \end{bmatrix} \text{ ("sat")} \quad (13.41)$$

Notice that multiplying $\mathbf{W}^E$ by a one-hot vector simply **selects the corresponding column** of $\mathbf{W}^E$. The embedding for “the” is column 1 of $\mathbf{W}^E$, the embedding for “cat” is column 2, and so on. In practice, this column selection (lookup) is how embeddings are implemented, rather than performing the full matrix multiplication.

The resulting input matrix for the sentence “the cat sat” (with $T = 3$ time steps) is:

$$\mathbf{P} = \begin{bmatrix} \mathbf{p}(1) & \mathbf{p}(2) & \mathbf{p}(3) \end{bmatrix} = \begin{bmatrix} 0.2 & 0.8 & -0.1 \\ -0.5 & 0.3 & 0.7 \\ 0.1 & -0.4 & 0.2 \end{bmatrix} \quad (13.42)$$

(dimension $R \times T = 3 \times 3$). This is the matrix $\mathbf{P}$ that is used as input to the self-attention operation.

The embedding matrix $\mathbf{W}^E$ is trained along with all the other weights in the network. During training, the columns of $\mathbf{W}^E$ are adjusted so that words with similar meanings end up with similar embedding vectors. For example, the vectors for “cat” and “dog” would be closer together (as measured by dot product or cosine similarity) than the vectors for “cat” and “sat.” This is directly related to the inner product similarity used in the attention operation (Eq. 13.21).

Embeddings can also be pretrained outside of the network, as in the Word2Vec method ([Mikolov et al., 2013]).

Seq2Seq and Attention

Positional Encoding

As noted previously, self-attention is permutation invariant: the dot product between two tokens depends only on their embedding vectors, not on where they appear in the sequence. For example, if we swap the positions of “cat” and “sat” in the input, the embedding vectors would also be swapped, but the set of dot products would remain the same. The sentences “the cat sat” and “the sat cat” would produce identical attention outputs (with the corresponding outputs also swapped). Clearly, word order matters, and the network needs some way to distinguish between different positions.

The solution proposed in [Vaswani et al., 2017] is to add a position-dependent vector to each embedding before it enters the attention operation:

$$\tilde{\mathbf{p}}(t) = \mathbf{p}(t) + \mathbf{p}^e(t) \quad (13.43)$$

where $\mathbf{p}(t)$ is the embedding vector from Eq. 13.37 and $\mathbf{p}^e(t)$ is the *positional encoding* vector for position t . The modified input $\tilde{\mathbf{p}}(t)$ is used in place of $\mathbf{p}(t)$ in the self-attention operation. Because each position has a unique encoding vector, the attention mechanism can now distinguish between tokens at different positions, even if the tokens themselves are the same.

The positional encoding vectors must satisfy two properties: each position should have a unique encoding, and the encoding should allow the network to easily learn to attend to relative positions (e.g., “the word two positions to the left”). The sinusoidal encoding of [Vaswani et al., 2017] achieves this by using sine and cosine functions at different frequencies. For an embedding of dimension R , the elements of the positional encoding vector are defined as

$$p_{2i}^e(t) = \sin\left(\frac{t}{10000^{2i/R}}\right) \quad (13.44)$$

$$p_{2i+1}^e(t) = \cos\left(\frac{t}{10000^{2i/R}}\right) \quad (13.45)$$

where $i = 0, 1, \dots$ indexes the dimension pairs. Each pair of dimensions $(2i, 2i+1)$ corresponds to a sinusoid with a different frequency. The even-indexed elements use sine and the odd-indexed elements use cosine. The frequencies decrease geometrically from 1 (for $i = 0$) to $1/10000$ (for $i = R/2 - 1$).

The reason for using both sine and cosine is that any fixed offset Δt can be represented as a linear transformation of the encoding: $\mathbf{p}^e(t + \Delta t) = \mathbf{M}_{\Delta t} \mathbf{p}^e(t)$, where $\mathbf{M}_{\Delta t}$ is a block-diagonal matrix of 2×2 rotation matrices. This makes it easy for the network to learn to attend to relative positions.

The factor 10000 is an empirical choice that determines the range of positional wavelengths.

Consider the embedding vectors from the previous example, where $R = 3$. Because the dimension is odd, we apply one complete sine/cosine pair for $i = 0$ (covering dimensions 0 and 1) and a single sine for $i = 1$ (covering dimension 2):

$$\begin{array}{c} 2 \\ +2 \\ \hline 4 \end{array}$$

$$p_0^e(t) = \sin(t) \quad (13.46)$$

$$p_1^e(t) = \cos(t) \quad (13.47)$$

$$p_2^e(t) = \sin\left(\frac{t}{10000^{2/3}}\right) = \sin\left(\frac{t}{464.2}\right) \quad (13.48)$$

Computing the positional encoding vectors for $t = 1, 2, 3$:

$$\mathbf{p}^e(1) = \begin{bmatrix} \sin(1) \\ \cos(1) \\ \sin(0.0022) \end{bmatrix} = \begin{bmatrix} 0.841 \\ 0.540 \\ 0.002 \end{bmatrix}, \quad \mathbf{p}^e(2) = \begin{bmatrix} \sin(2) \\ \cos(2) \\ \sin(0.0043) \end{bmatrix} = \begin{bmatrix} 0.909 \\ -0.416 \\ 0.004 \end{bmatrix}$$

$$\mathbf{p}^e(3) = \begin{bmatrix} \sin(3) \\ \cos(3) \\ \sin(0.0065) \end{bmatrix} = \begin{bmatrix} 0.141 \\ -0.990 \\ 0.006 \end{bmatrix}$$

Notice that the first two elements (the high-frequency pair) vary rapidly with position, while the third element (the low-frequency component) changes very slowly. In a full-sized model with $R = 512$, there would be 256 frequency pairs, ranging from rapid oscillation to extremely slow variation, providing a rich position signal at many scales.

Now we add the positional encodings to the embedding vectors from Eq. 13.42:

$$\begin{aligned} \tilde{\mathbf{p}}(1) &= \mathbf{p}(1) + \mathbf{p}^e(1) = \begin{bmatrix} 0.2 \\ -0.5 \\ 0.1 \end{bmatrix} + \begin{bmatrix} 0.841 \\ 0.540 \\ 0.002 \end{bmatrix} = \begin{bmatrix} 1.041 \\ 0.040 \\ 0.102 \end{bmatrix} \quad (\text{"the", pos 1}) \\ \tilde{\mathbf{p}}(2) &= \mathbf{p}(2) + \mathbf{p}^e(2) = \begin{bmatrix} 0.8 \\ 0.3 \\ -0.4 \end{bmatrix} + \begin{bmatrix} 0.909 \\ -0.416 \\ 0.004 \end{bmatrix} = \begin{bmatrix} 1.709 \\ -0.116 \\ -0.396 \end{bmatrix} \quad (\text{"cat", pos 2}) \\ \tilde{\mathbf{p}}(3) &= \mathbf{p}(3) + \mathbf{p}^e(3) = \begin{bmatrix} -0.1 \\ 0.7 \\ 0.2 \end{bmatrix} + \begin{bmatrix} 0.141 \\ -0.990 \\ 0.006 \end{bmatrix} = \begin{bmatrix} 0.041 \\ -0.290 \\ 0.206 \end{bmatrix} \quad (\text{"sat", pos 3}) \end{aligned}$$

Seq2Seq and Attention

The modified input matrix is:

$$\tilde{\mathbf{P}} = \begin{bmatrix} \tilde{\mathbf{p}}(1) & \tilde{\mathbf{p}}(2) & \tilde{\mathbf{p}}(3) \end{bmatrix} = \begin{bmatrix} 1.041 & 1.709 & 0.041 \\ 0.040 & -0.116 & -0.290 \\ 0.102 & -0.396 & 0.206 \end{bmatrix} \quad (13.49)$$

This matrix $\tilde{\mathbf{P}}$ is used in place of $\mathbf{P}$ in the self-attention equations. Now, if we were to swap “cat” and “sat” in the input (placing “cat” at position 3 and “sat” at position 2), the positional encoding vectors would be different: “cat” would receive $\mathbf{p}^e(3)$ instead of $\mathbf{p}^e(2)$, and “sat” would receive $\mathbf{p}^e(2)$ instead of $\mathbf{p}^e(3)$. This breaks the permutation invariance and allows the network to distinguish between different word orderings.

To illustrate this, the dot product between the position-encoded representations of “the” and “cat” is:

$$\begin{aligned} \tilde{\mathbf{p}}(1)^T \tilde{\mathbf{p}}(2) &= (1.041)(1.709) + (0.040)(-0.116) + (0.102)(-0.396) \\ &= 1.780 - 0.005 - 0.040 = 1.735 \end{aligned}$$

If we swap the positions of “cat” and “sat,” then “cat” would be at position 3 and its modified embedding would be:

$$\mathbf{p}(\text{“cat”}) + \mathbf{p}^e(3) = \begin{bmatrix} 0.8 \\ 0.3 \\ -0.4 \end{bmatrix} + \begin{bmatrix} 0.141 \\ -0.990 \\ 0.006 \end{bmatrix} = \begin{bmatrix} 0.941 \\ -0.690 \\ -0.394 \end{bmatrix}$$

The dot product of “the” (still at position 1) with “cat” (now at position 3) would be:

$$\begin{aligned} \tilde{\mathbf{p}}(1)^T \tilde{\mathbf{p}}(2) &= (1.041)(0.941) + (0.040)(-0.690) + (0.102)(-0.394) \\ &= 0.980 - 0.028 - 0.040 = 0.912 \end{aligned}$$

The dot product changed from 1.734 to 0.912, which means the attention weights will be different. The network can learn to use these differences to capture the importance of word order.

An alternative to the fixed sinusoidal encoding is to make the positional encoding vectors trainable parameters, similar to the embedding matrix. Both approaches have been used successfully, although the sinusoidal encoding has the advantage of potentially

generalizing to sequence lengths not seen during training. Another type of encoding, called Rotary Position Embedding (RoPE, see [Su et al., 2024]), encodes position by rotating the query and key vectors in the attention mechanism.

Full Decoder-Only Transformer

We can put together the various components we have discussed to obtain the basic structure of a full decoder-only transformer network, as shown in Figure 13.11. The input passes through the embedding operation, after which the position encoding is added. This goes into the self-attention operation, which is then followed by a couple of fully connected layers.

The transformer generally has multi-head attention and skip connections across the attention section and across the MLP section. It can also have a normalization block after the attention and after the MLP.

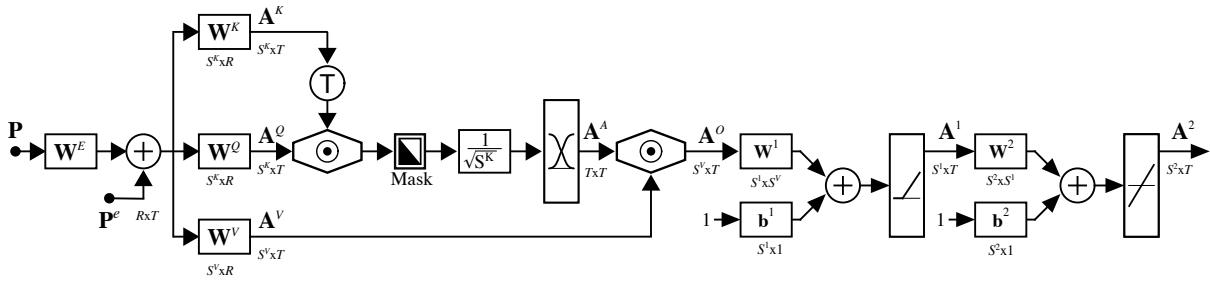


Figure 13.11: Decoder-only Transformer

Recall that the input $\mathbf{P}$ is formed by a horizontally stacked TDL of length T from the sequence $\mathbf{p}(t)$. Because of the horizontal stacking, each $\mathbf{p}(t)$ travels individually through the network, except at the attention operation, where the time steps are mixed. This means that the time steps are not mixed when passing through the fully connected layers. Although we showed in Chapter 11 of NND2 that a two-layer fully connected network can create virtually any function of interest, given enough neurons in the hidden layer, the network in Figure 13.11 cannot create an arbitrary function of the T values of $\mathbf{p}(t)$ that are in $\mathbf{P}$. This is unlike the FTDNN of Figure 12.1, in which a vertically stacked TDL is input to a multilayer network. With the vertical TDL, the time steps are mixed within the fully connected layers, and therefore the network can theoretically create an arbitrary function of all $\mathbf{p}(t)$ in the TDL with just two layers.

Seq2Seq and Attention

In order to be able to be a universal approximator, the transformer network needs to have at least two transformer blocks, where one block has the self-attention operation followed by a two-layer MLP. (See [Yun et al., 2019]) Figure 13.11 has one transformer block.

The decoder-only transformer is the core of GPT (Generative Pre-trained Transformer) models, like ChatGPT, Claude, DeepSeek, etc. Notice that, because of the horizontal TDL at the input, the number of columns (T) remains constant at each stage of the network. If the columns in the target sequence represent tokens that follow each token in the input sequence, then the transformer becomes a next-step predictor, if masked attention is used. This means that, unlike with the FTDNN, with the vertical TDL, the same network weights and biases can be used, even when the length of the input sequence, T , changes. Solved Problem P13.2 demonstrates how the transformer can be used to generate sequences.

Epilogue

The concept of attention was developed as a way to improve the performance of sequence-to-sequence models, which originally used RNNs in both the encoder and decoder. Attention was a particular way to include information in the encoder context that was not efficiently contained in the RNN state. The most popular form of attention used the dot product between the most recent decoder state and all previous encoder states in order to determine how much of each previous state to be included in the final encoder context. It was later shown that the RNN networks could be dropped and replaced entirely with a self-attention operation.

Using dot product attention was especially useful for natural language processing when using embeddings to encode input tokens. These embeddings naturally placed related words in the same region of the embedding space. The dot products between embedding vectors thus provided useful information about the relationship between the tokens. Another important property of the transformer model was that, unlike RNNs, the computation could be parallelized, and therefore the GPU training of transformer networks was much more efficient than RNNs.

Seq2Seq and Attention

Further Reading

[Sutskever et al., 2014] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. *Advances in neural information processing systems*, 27, 2014

This paper introduced the sequence-to-sequence model. The input sequence is applied to an encoder subnetwork (an LSTM network in this original paper). The encoder then produces a context vector that is passed to a decoder network (also an LSTM), which then generates the output sequence. The application of the seq2seq model in this paper is language translation.

[Bahdanau et al., 2014] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014

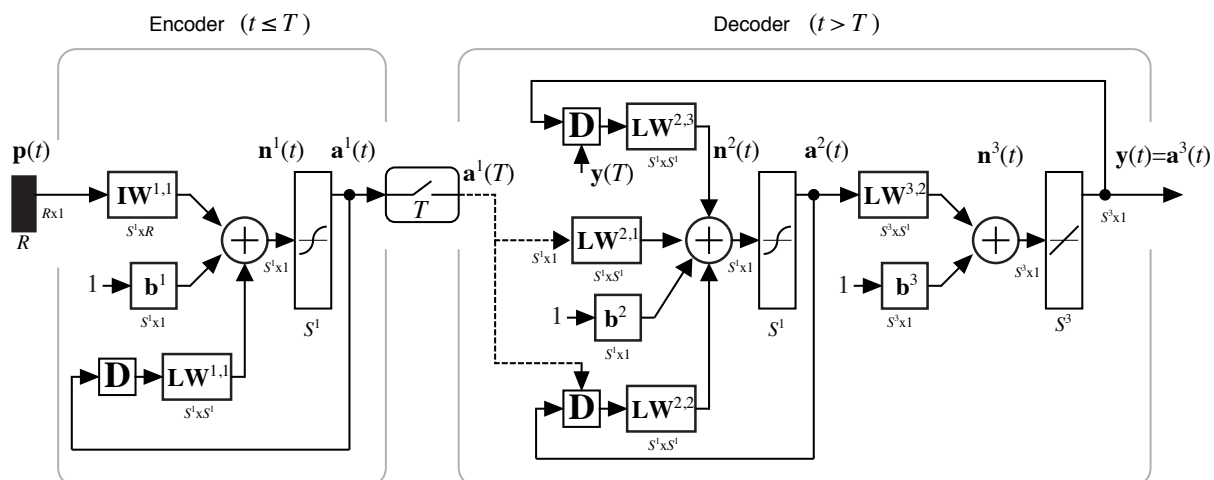
This paper was the first to propose the addition of attention between the encoder and decoder subnetworks of the seq2seq model. The attention mechanism gave the model a greater ability to efficiently look back over previous items in the input sequence so as to improve the output sequence.

[Vaswani et al., 2017] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017

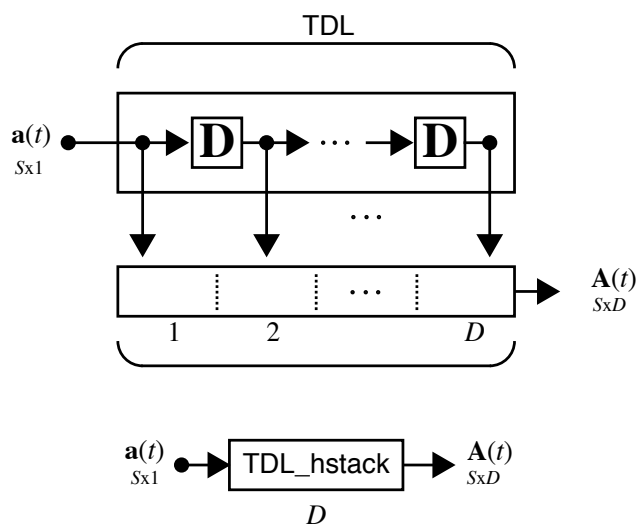
In this paper the authors proposed removing the recurrent networks in the encoder and decoder subnetworks of the seq2seq model and using only attention. This produced the transformer network. With the recurrent connections removed, the attention-only model could be implemented in parallel and enabled massive scaling – leading to ChatGPT and associated large language models.

Summary of Results

Sequence to Sequence Network

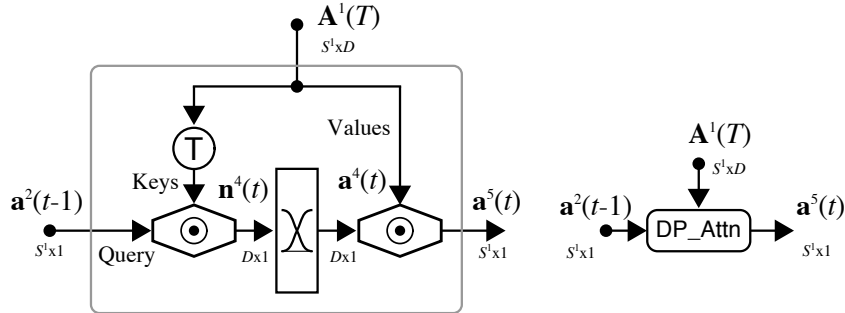


Horizontal Stacked Tapped Delay Line

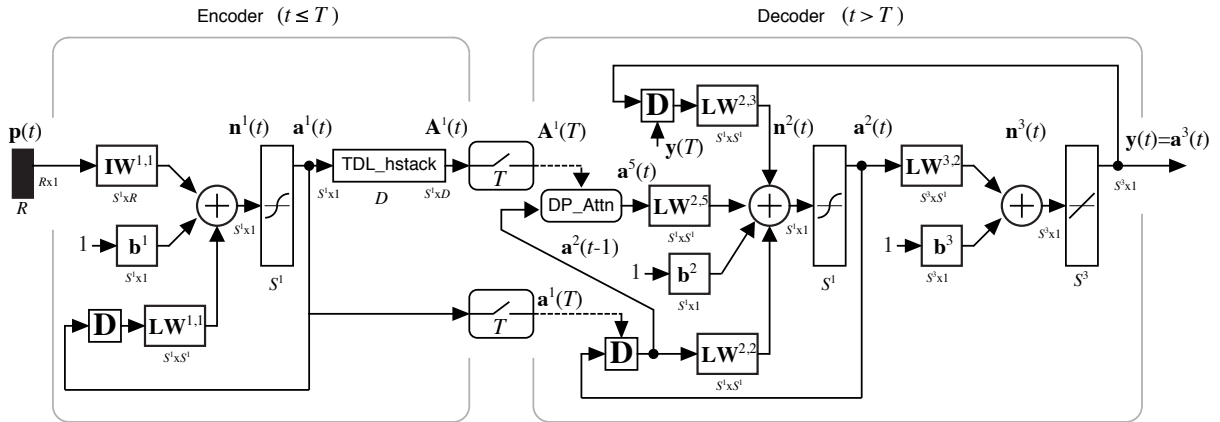


Seq2Seq and Attention

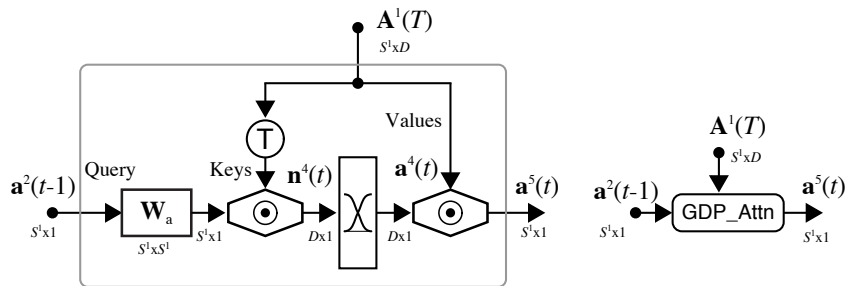
Dot Product Attention



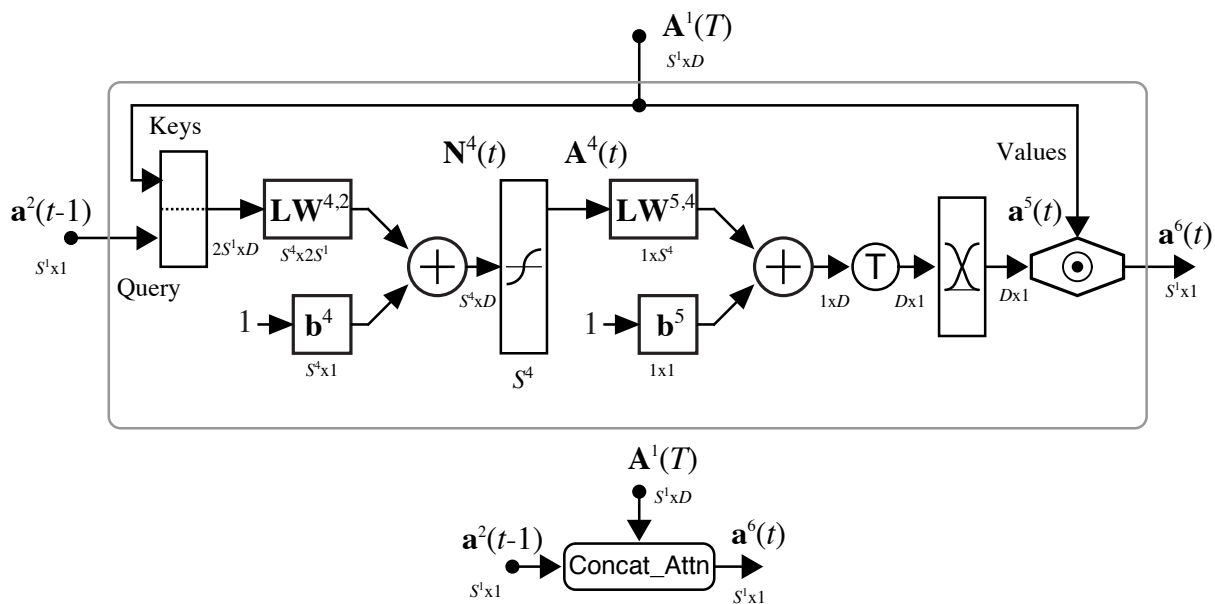
Sequence to Sequence Network With Attention



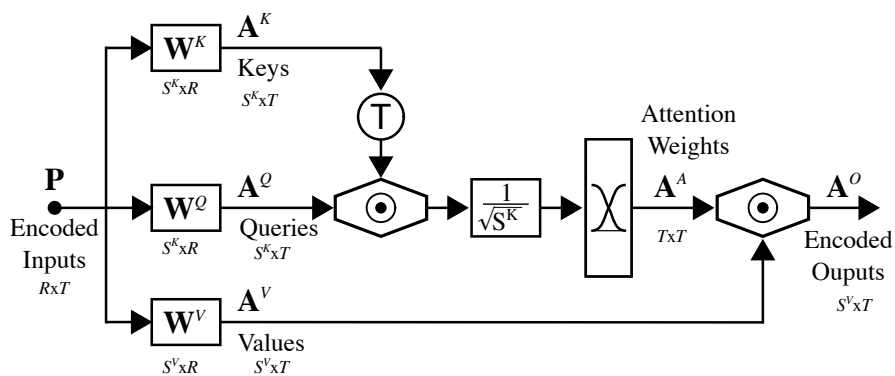
General Dot Product Attention



Concatenation Attention

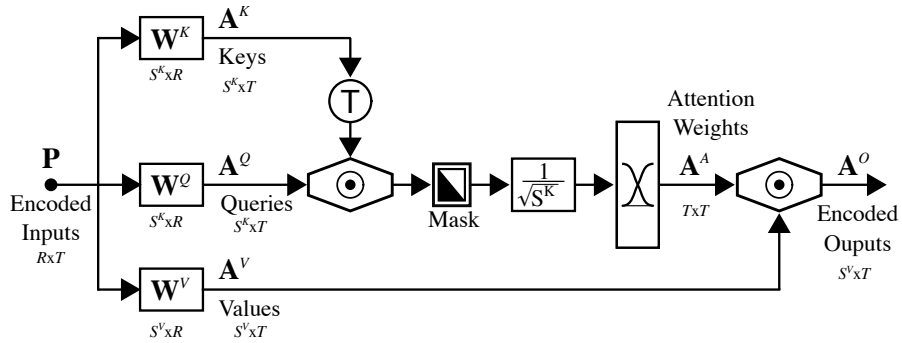


Self Dot Product Attention

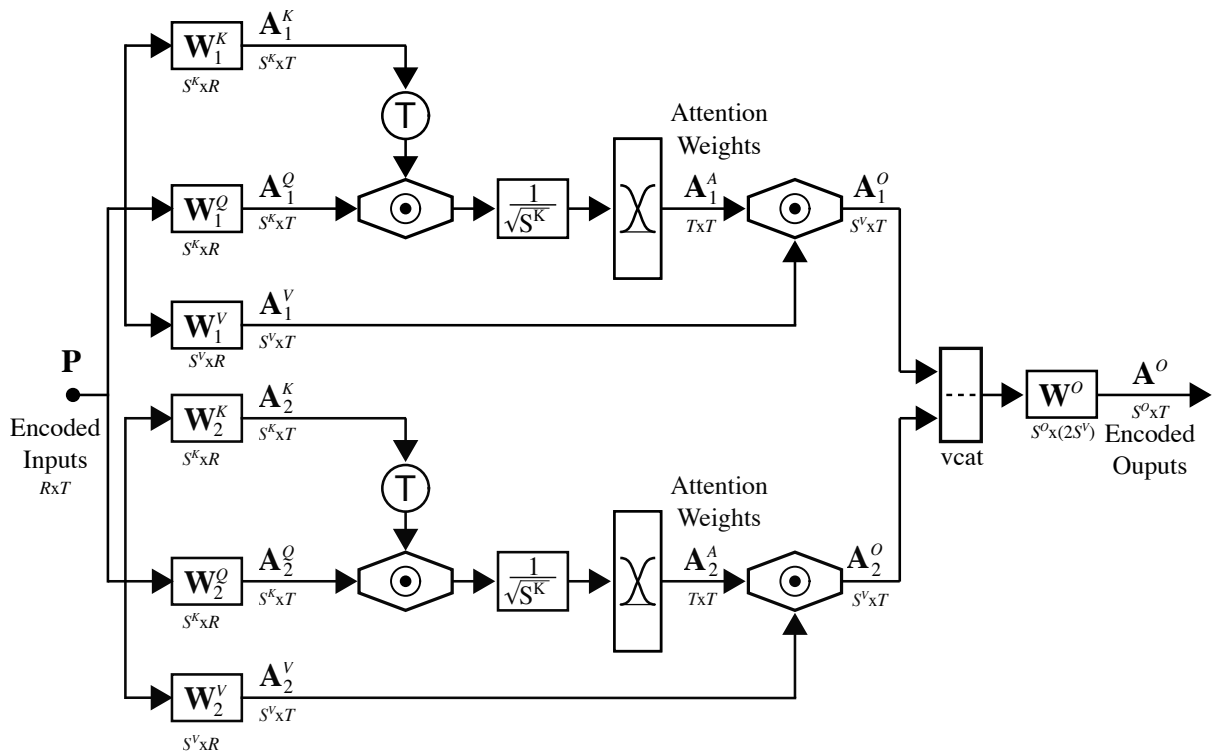


Seq2Seq and Attention

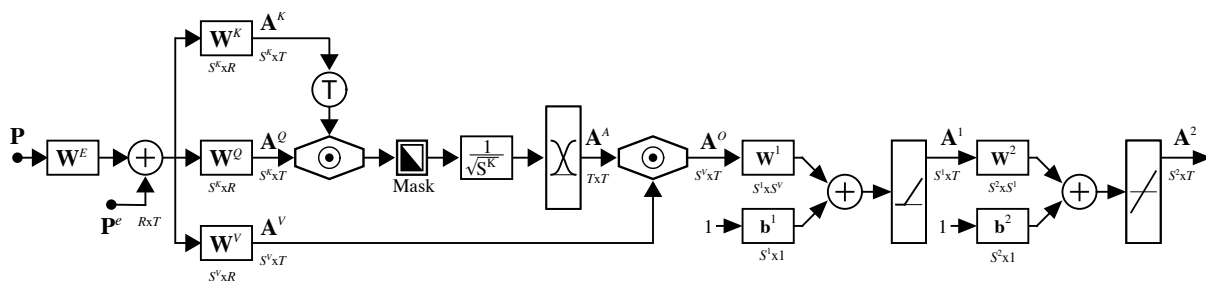
Masked Attention



Multi-head Attention



Decoder-only Transformer



Seq2Seq and Attention

Solved Problems

P13.1 Use a numerical example to demonstrate how masked attention operates.

Let the input sequence be the following:

$$\mathbf{P} = \begin{bmatrix} \mathbf{p}(1) & \mathbf{p}(2) & \mathbf{p}(3) \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

and let the query, key and value matrices be

$$\mathbf{W}^Q = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \mathbf{W}^K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}, \mathbf{W}^V = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

The first step is to multiply the input sequence by these matrices.

$$\mathbf{A}^Q = \mathbf{W}^Q \mathbf{P} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 2 \\ 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} \mathbf{a}^Q(1) & \mathbf{a}^Q(2) & \mathbf{a}^Q(3) \end{bmatrix}$$

$$\mathbf{A}^K = \mathbf{W}^K \mathbf{P} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 2 & 1 \\ 2 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{a}^K(1) & \mathbf{a}^K(2) & \mathbf{a}^K(3) \end{bmatrix}$$

$$\mathbf{A}^V = \mathbf{W}^V \mathbf{P} = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{a}^V(1) & \mathbf{a}^V(2) & \mathbf{a}^V(3) \end{bmatrix}$$

Now we perform the attention dot product operation.

$$\begin{bmatrix} \mathbf{A}^K \end{bmatrix}^T \mathbf{A}^Q = \begin{bmatrix} 0 & 2 \\ 2 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 2 \\ 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 \\ 2 & 4 & 4 \\ 2 & 2 & 2 \end{bmatrix}$$

Next we set up the mask. Since we don't want the network to look ahead, the mask will be upper triangular:

$$\mathbf{M} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{No token attends to future tokens}$$

When we operate with the mask, the inner products that are not to be considered are set to $-\infty$:

$$\mathbf{M} \boxtimes [\mathbf{A}^K]^T \mathbf{A}^Q = \begin{bmatrix} 2 & 0 & 0 \\ -\infty & 4 & 4 \\ -\infty & -\infty & 2 \end{bmatrix}$$

The complete operation with normalization produces the following:

$$\mathbf{N}^A = \frac{1}{\sqrt{2}} \mathbf{M} \boxtimes [\mathbf{A}^K]^T \mathbf{A}^Q = \begin{bmatrix} \sqrt{2} & 0 & 0 \\ -\infty & 2\sqrt{2} & 2\sqrt{2} \\ -\infty & -\infty & \sqrt{2} \end{bmatrix}$$

Passing through the softmax function produces the final attention weights:

$$\mathbf{A}^A = \text{softmax}(\mathbf{N}^A, \text{columns}) = \begin{bmatrix} 1 & 0.06 & 0.04 \\ 0 & 0.94 & 0.77 \\ 0 & 0 & 0.19 \end{bmatrix}$$

Finally, multiplying the value vectors by the attention weights produces the output sequence:

$$\begin{aligned} \mathbf{A}^O = \mathbf{A}^V \mathbf{A}^A &= \begin{bmatrix} \mathbf{a}^V(1) & 0.06\mathbf{a}^V(1) + 0.94\mathbf{a}^V(2) & 0.04\mathbf{a}^V(1) + 0.77\mathbf{a}^V(2) + 0.19\mathbf{a}^V(3) \end{bmatrix} \\ &= \begin{bmatrix} 1 & 1.94 & 1.77 \\ 2 & 0.11 & 0.28 \end{bmatrix} \quad \begin{array}{l} \text{First column attends to first token.} \\ \text{Second column attends to first two tokens.} \end{array} \end{aligned}$$

P13.2 Demonstrate how the transformer can be used to generate an output sequence of arbitrary length from an input sequence of any length. Discuss how this network would be trained and how it would be used during inference.

Seq2Seq and Attention

Because the transformer uses a horizontal TDL of the input sequence as the input to the network, each column of the input to the network and the output of the network represents a single time step. If we want the transformer to be a one-step predictor, we can choose the target sequences during training to be the same as the input sequences, but shifted by one step. For example, if the transformer input is

$$\mathbf{P} = \begin{bmatrix} \mathbf{p}(1) & \mathbf{p}(2) & \mathbf{p}(3) \end{bmatrix}$$

then we would choose the target to be

$$\mathbf{T} = \begin{bmatrix} \mathbf{p}(2) & \mathbf{p}(3) & \mathbf{p}(4) \end{bmatrix}$$

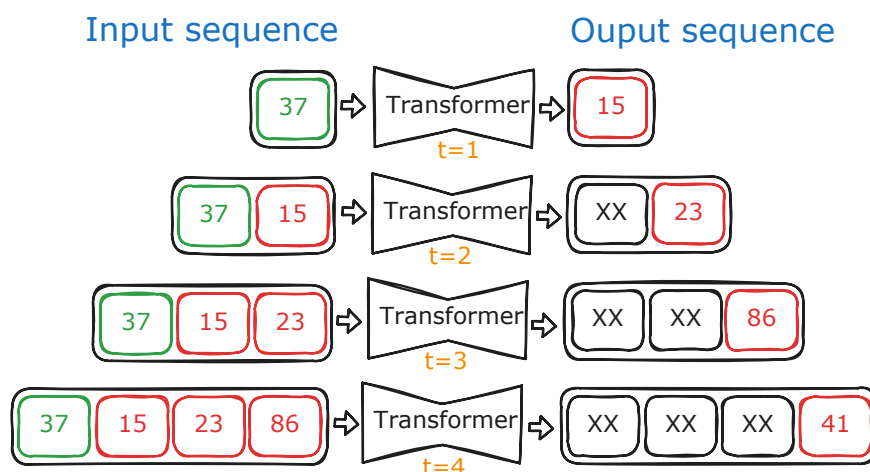
Since we are using masked attention, the network output in the first column, which has a target of $\mathbf{p}(2)$, only has access to the first input $\mathbf{p}(1)$. The network output in the second column, which has a target of $\mathbf{p}(3)$, only has access to the first two inputs $\mathbf{p}(1)$ and $\mathbf{p}(2)$. The network is trained to be a one-step-ahead predictor.

Now, once the network is trained, how can we use it to generate output sequences of length longer than the input sequence? First, consider the network diagram of Figure 13.11. Notice that the number of columns in the output always matches the number of columns in the input, and this is unrelated to the dimensions of any of the weights or biases in the network. (Note that the biases are added using broadcasting, so they are expanded to match the number of columns in the input.) This means that the network input sequences can be of arbitrary length. We can use this to generate arbitrary length sequences, as shown in the following figure.

At the first step the input is just $\mathbf{p}(1)$ (represented by the 37). The output of the network is the predicted next time step $\hat{\mathbf{p}}(2)$ (represented by the 15). If we want to predict two time steps ahead, we can form a new input as

$$\begin{bmatrix} \mathbf{p}(1) & \hat{\mathbf{p}}(2) \end{bmatrix}$$

which is represented by 37, 15 in the figure below.



The output of the network at the second time step would be

$$\begin{bmatrix} \hat{\mathbf{p}}(2) & \hat{\mathbf{p}}(3) \end{bmatrix}$$

Since we already have a $\hat{\mathbf{p}}(2)$, that term is discarded, but we can add $\hat{\mathbf{p}}(3)$ (represented by 23) to $\mathbf{p}(1)$ and $\hat{\mathbf{p}}(2)$ from the previous time step to create our next input.

$$\begin{bmatrix} \mathbf{p}(1) & \hat{\mathbf{p}}(2) & \hat{\mathbf{p}}(3) \end{bmatrix}$$

As shown in the figure, this process can continue indefinitely. In natural language processing, we stop when the network generates an end of file token.

The reason we can put an arbitrary length input sequence into the same network is that the only time the columns intermix is in the dot product operation, for which the same equation is used no matter how many tokens in the input sequence. This brings up an interesting question. If the network has been trained only with sequences that have some maximum length, will the network perform accurately when longer sequences are input to the network?

This is something to be concerned about. One issue is the positional encoding. With sinusoidal encoding, the formula can produce a vector for any position t , so there is no hard failure. But the model has never seen those positions during training, so it has no experience using them. Also, during training the model learned to form

Seq2Seq and Attention

useful attention patterns over at most $T_{\max}$ keys. With many more keys, the softmax tends to spread its weight more diffusely, which can cause the attention outputs to degrade in quality. There are techniques to ameliorate these issues, but it is something to be aware of.

P13.3 Use a numerical example to demonstrate multi-head attention with $H = 2$ heads. Show how the two heads produce different attention patterns, and combine them to form the final output.

Consider a three-token input sequence with embedding dimension $R = 4$:

$$\mathbf{P} = [\mathbf{p}(1) \quad \mathbf{p}(2) \quad \mathbf{p}(3)] = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 2 & 1 \\ 2 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$$

With $H = 2$ heads, each head operates in a $S^K = R/H = 2$ dimensional subspace. Each head has its own query, key and value matrices (2×4).

Head 1 uses the following weight matrices:

$$\mathbf{W}_1^Q = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{W}_1^K = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{W}_1^V = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$\mathbf{W}_1^Q$ extracts the first two dimensions as the query, $\mathbf{W}_1^K$ swaps the first two dimensions as the key, and $\mathbf{W}_1^V$ combines the first and third dimensions as the value. The projected queries, keys and values for head 1 are

$$\mathbf{A}_1^Q = \mathbf{W}_1^Q \mathbf{P} = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 2 & 1 \end{bmatrix}, \quad \mathbf{A}_1^K = \mathbf{W}_1^K \mathbf{P} = \begin{bmatrix} 0 & 2 & 1 \\ 1 & 0 & -1 \end{bmatrix}, \quad \mathbf{A}_1^V = \mathbf{W}_1^V \mathbf{P} = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 2 & 1 \end{bmatrix}$$

The scaled dot products are

$$\mathbf{N}_1^A = \frac{1}{\sqrt{S^K}} [\mathbf{A}_1^K]^T \mathbf{A}_1^Q = \frac{1}{\sqrt{2}} \begin{bmatrix} 0 & 2 & 1 \\ 2 & 0 & -2 \\ 1 & -2 & -2 \end{bmatrix} = \begin{bmatrix} 0 & \sqrt{2} & \frac{1}{\sqrt{2}} \\ \sqrt{2} & 0 & -\sqrt{2} \\ \frac{1}{\sqrt{2}} & -\sqrt{2} & -\sqrt{2} \end{bmatrix}$$

Passing each column through the softmax gives the attention weights for head 1:

$$\mathbf{A}_1^A = \text{softmax}(\mathbf{N}_1^A, \text{columns}) = \begin{bmatrix} 0.14 & 0.77 & 0.81 \\ 0.58 & 0.19 & 0.10 \\ 0.28 & 0.05 & 0.10 \end{bmatrix}$$

Head 1 attention pattern: Token 1 attends most to token 2 (weight 0.58). Token 2 attends strongly to token 1 (weight 0.77). Token 3 attends primarily to token 1 (weight 0.81). This head captures a “cross-attention” relationship, where tokens attend primarily to *other* tokens rather than to themselves.

The output of head 1 is

$$\mathbf{A}_1^O = \mathbf{A}_1^V \mathbf{A}_1^A = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 2 & 1 \end{bmatrix} \begin{bmatrix} 0.14 & 0.77 & 0.81 \\ 0.58 & 0.19 & 0.10 \\ 0.28 & 0.05 & 0.10 \end{bmatrix} = \begin{bmatrix} 0.42 & 2.30 & 2.42 \\ 1.44 & 0.42 & 0.29 \end{bmatrix}$$

Head 2 uses the following weight matrices:

$$\mathbf{W}_2^Q = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{W}_2^K = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad \mathbf{W}_2^V = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

$\mathbf{W}_2^Q$ extracts the third and fourth dimensions as the query, $\mathbf{W}_2^K$ swaps those dimensions as the key, and $\mathbf{W}_2^V$ combines the second, third and fourth dimensions as the value. The projected queries, keys and values for head 2 are

$$\mathbf{A}_2^Q = \mathbf{W}_2^Q \mathbf{P} = \begin{bmatrix} 2 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}, \quad \mathbf{A}_2^K = \mathbf{W}_2^K \mathbf{P} = \begin{bmatrix} 1 & 1 & 0 \\ 2 & 0 & 1 \end{bmatrix}, \quad \mathbf{A}_2^V = \mathbf{W}_2^V \mathbf{P} = \begin{bmatrix} 2 & 0 & 1 \\ 1 & 3 & 1 \end{bmatrix}$$

The scaled dot products are

$$\mathbf{N}_2^A = \frac{1}{\sqrt{2}} [\mathbf{A}_2^K]^T \mathbf{A}_2^Q = \frac{1}{\sqrt{2}} \begin{bmatrix} 4 & 2 & 1 \\ 2 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 2\sqrt{2} & \sqrt{2} & \frac{1}{\sqrt{2}} \\ \sqrt{2} & 0 & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \end{bmatrix}$$

The attention weights for head 2 are

Seq2Seq and Attention

$$\mathbf{A}_2^A = \text{softmax}(\mathbf{N}_2^A, \text{columns}) = \begin{bmatrix} 0.73 & 0.58 & 0.40 \\ 0.18 & 0.14 & 0.40 \\ 0.09 & 0.28 & 0.20 \end{bmatrix}$$

Head 2 attention pattern: Token 1 attends predominantly to itself (weight 0.73). Token 2 attends most to token 1 (weight 0.58). Token 3 splits its attention evenly between tokens 1 and 2 (both 0.40). This head captures a “self-and-neighbor” pattern — quite different from head 1’s cross-attention pattern.

The output of head 2 is

$$\mathbf{A}_2^O = \mathbf{A}_2^V \mathbf{A}_2^A = \begin{bmatrix} 2 & 0 & 1 \\ 1 & 3 & 1 \end{bmatrix} \begin{bmatrix} 0.73 & 0.58 & 0.40 \\ 0.18 & 0.14 & 0.40 \\ 0.09 & 0.28 & 0.20 \end{bmatrix} = \begin{bmatrix} 1.56 & 1.44 & 1.00 \\ 1.36 & 1.28 & 1.80 \end{bmatrix}$$

Concatenation and output projection. Each head output has dimension $S^K = 2$. We vertically concatenate the two head outputs to form a 4×3 matrix (dimension $R \times T$), which restores the original embedding dimension:

$$\mathbf{A}_{\text{concat}}^O = \begin{bmatrix} \mathbf{A}_1^O \\ \mathbf{A}_2^O \end{bmatrix} = \begin{bmatrix} 0.42 & 2.30 & 2.42 \\ 1.44 & 0.42 & 0.29 \\ 1.56 & 1.44 & 1.00 \\ 1.36 & 1.28 & 1.80 \end{bmatrix}$$

In practice, a learned output projection matrix $\mathbf{W}^O$ ($R \times R$) is applied to mix the head outputs: $\mathbf{A}^O = \mathbf{W}^O \mathbf{A}_{\text{concat}}^O$. For this example we use $\mathbf{W}^O = \mathbf{I}$, so $\mathbf{A}^O = \mathbf{A}_{\text{concat}}^O$.

Comparison of the two heads. The key point of multi-head attention is that each head can learn to attend to different relationships in the data:

	Head 1	Head 2
Token 1 attends to	Token 2 (0.58)	Token 1 (0.73)
Token 2 attends to	Token 1 (0.77)	Token 1 (0.58)
Token 3 attends to	Token 1 (0.81)	Tokens 1 & 2 (0.40, 0.40)

Head 1 primarily performs cross-attention — tokens attend to other tokens — while head 2 gives more weight to self-attention and nearby context. In a real transformer, these different patterns might correspond to different linguistic relationships: one head might resolve pronoun references, while another tracks syntactic structure. The output projection $\mathbf{W}^O$ then combines these complementary views into a single representation.

Seq2Seq and Attention

Exercises

E13.1 Consider the dot product attention of Eqs. 13.7–13.9. The encoder has produced the following horizontally stacked state matrix and the previous decoder state is given below.

$$\mathbf{A}^1(T) = \begin{bmatrix} 0.6 & -0.3 & 0.5 \\ 0.2 & 0.8 & -0.4 \\ -0.5 & 0.1 & 0.3 \end{bmatrix}, \quad \mathbf{a}^2(t-1) = \begin{bmatrix} 0.3 \\ -0.2 \\ 0.5 \end{bmatrix}$$

- i. Compute the inner product vector $\mathbf{n}^4(t) = [\mathbf{A}^1(T)]^T \mathbf{a}^2(t-1)$.
- ii. Compute the attention weights $\mathbf{a}^4(t) = \text{softmax}(\mathbf{n}^4(t))$.
- iii. Compute the context vector $\mathbf{a}^5(t) = \mathbf{A}^1(T) \mathbf{a}^4(t)$.
- iv. Which encoder state contributes most to the context vector? Why?

E13.2 This exercise demonstrates the permutation invariance of self-attention and shows how positional encoding breaks it. Use the embedding matrix and positional encoding formulas from the examples in this chapter ($R = 3$, $V = 5$, with the same $\mathbf{W}^E$).

- i. Use the embedding matrix $\mathbf{W}^E$ from Eq. 13.38 to find the embedding vectors for “cat” and “sat.” Compute the dot product $\mathbf{p}(\text{cat})^T \mathbf{p}(\text{sat})$ without any positional encoding.
- ii. Now consider two orderings of the two-token input: (i) “cat” at position 1 and “sat” at position 2; and (ii) “sat” at position 1 and “cat” at position 2. Without positional encoding, what is the dot product $\mathbf{p}(1)^T \mathbf{p}(2)$ in each case? What does this tell you about permutation invariance?
- iii. Now add sinusoidal positional encodings. Compute $\mathbf{p}^e(1)$ and $\mathbf{p}^e(2)$ using Eqs. 13.44 and 13.45. Then compute the dot product $\tilde{\mathbf{p}}(1)^T \tilde{\mathbf{p}}(2)$ for each ordering and verify that the two dot products are now different.

E13.3 Perform the self-attention operation for the following three-token input sequence, using identity weight matrices $\mathbf{W}^Q = \mathbf{W}^K = \mathbf{W}^V = \mathbf{I}$.

$$\mathbf{P} = \begin{bmatrix} \mathbf{p}(1) & \mathbf{p}(2) & \mathbf{p}(3) \end{bmatrix} = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

- i. Since the weight matrices are identity, we have $\mathbf{A}^Q = \mathbf{A}^K = \mathbf{A}^V = \mathbf{P}$. Compute the scaled dot product matrix $\mathbf{N}^A = \frac{1}{\sqrt{3}} [\mathbf{A}^K]^T \mathbf{A}^Q$.
- ii. Compute the attention weight matrix $\mathbf{A}^A = \text{softmax}(\mathbf{N}^A, \text{columns})$.
- iii. Compute the output matrix $\mathbf{A}^O = \mathbf{A}^V \mathbf{A}^A$.
- iv. Note that $\mathbf{p}(1)$ and $\mathbf{p}(3)$ differ only in the sign of their third element, while $\mathbf{p}(2)$ is orthogonal to $\mathbf{p}(1)$ and $\mathbf{p}(3)$ in the first and third dimensions. Explain the pattern of attention weights you observe. In particular, why does $\mathbf{p}(1)$ attend strongly to itself and to $\mathbf{p}(3)$, while $\mathbf{p}(2)$ attends more evenly?

E13.4 This exercise illustrates why the scaling factor $1/\sqrt{S^K}$ is important in the attention operation. Consider a two-token input $\mathbf{P} = \mathbf{I}$ (the 2×2 identity matrix), with identity weight matrices $\mathbf{W}^Q = \mathbf{W}^K = \mathbf{W}^V = \mathbf{I}$.

- i. Compute the attention weights $\mathbf{A}^A$ with the standard scaling factor $1/\sqrt{S^K} = 1/\sqrt{2}$. What fraction of the attention does each token pay to itself?
- ii. Now suppose the input vectors are scaled up by a factor of c , so that $\mathbf{P} = c\mathbf{I}$. Write an expression for the dot products $[\mathbf{A}^K]^T \mathbf{A}^Q$ in terms of c .
- iii. Compute the attention weights for $c = 2$ and $c = 5$. What happens to the attention weights as c grows large? What does this mean for the gradients of the softmax function?
- iv. Explain why the $1/\sqrt{S^K}$ scaling factor mitigates this problem during training.

E13.5 Consider a model with embedding dimension $R = 4$.

- i. Write out the four components of the positional encoding vector $\mathbf{p}^e(t)$, identifying the frequency associated with each dimension pair.

Seq2Seq and Attention

- ii. Compute the positional encoding vectors $\mathbf{p}^e(1)$, $\mathbf{p}^e(2)$, $\mathbf{p}^e(3)$, and $\mathbf{p}^e(4)$.
- iii. Write the positional encoding vectors for $t = 1, \dots, 4$ as the columns of a 4×4 matrix $\mathbf{P}^e$. Notice that the rows correspond to different frequency components and the columns correspond to different positions. What pattern do you observe in the rows?
- iv. Compute the dot products $\mathbf{p}^e(1)^T \mathbf{p}^e(t)$ for $t = 1, 2, 3, 4$. What do you notice about how the dot product changes as the position distance $|t - 1|$ increases?

E13.6 This exercise works through a two-head self-attention operation. The input sequence has two tokens and embedding dimension $R = 4$.

$$\mathbf{P} = \begin{bmatrix} \mathbf{p}(1) & \mathbf{p}(2) \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

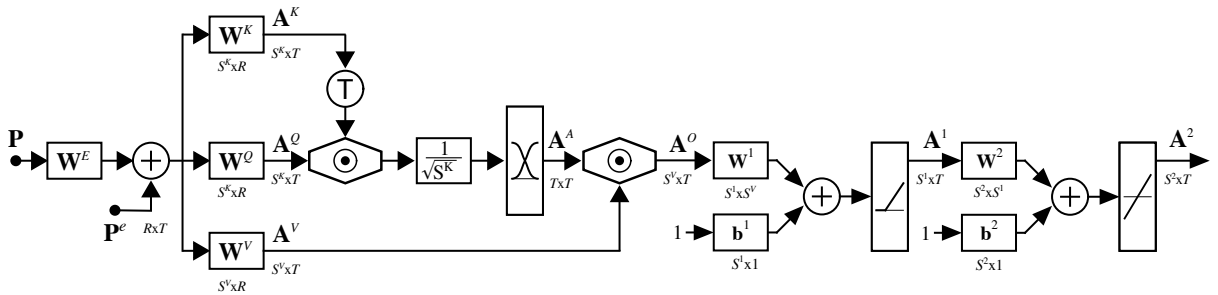
Each head operates in a 2-dimensional subspace ($S^K = R/H = 2$). Head 1 uses the first two dimensions and head 2 uses the last two dimensions:

$$\mathbf{W}_1^Q = \mathbf{W}_1^K = \mathbf{W}_1^V = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{W}_2^Q = \mathbf{W}_2^K = \mathbf{W}_2^V = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- i. Compute the queries, keys, and values for each head: $\mathbf{A}_h^Q = \mathbf{W}_h^Q \mathbf{P}$, $\mathbf{A}_h^K = \mathbf{W}_h^K \mathbf{P}$, $\mathbf{A}_h^V = \mathbf{W}_h^V \mathbf{P}$ for $h = 1, 2$.
- ii. Compute the scaled dot products $\mathbf{N}_h^A = \frac{1}{\sqrt{S^K}} [\mathbf{A}_h^K]^T \mathbf{A}_h^Q$, attention weights $\mathbf{A}_h^A = \text{softmax}(\mathbf{N}_h^A, \text{columns})$, and output $\mathbf{A}_h^O = \mathbf{A}_h^V \mathbf{A}_h^A$ for each head.
- iii. Form the final multi-head output by vertically concatenating the two head outputs: $\mathbf{A}^O = \begin{bmatrix} \mathbf{A}_1^O \\ \mathbf{A}_2^O \end{bmatrix}$. What is the dimension of this output?

- iv. In this example, the two heads receive different slices of the input. In practice, the weight matrices $\mathbf{W}_h^Q$, $\mathbf{W}_h^K$, $\mathbf{W}_h^V$ are trained freely and are not constrained to select specific dimensions. What does this allow each head to learn?

E13.7 Write out the backpropagation equations to compute the gradient for the weights and biases in the following network.



E13.8 Both the FTDNN from the previous chapter and the Transformer from this chapter can be used for sequence prediction. Suppose that you have a data set that consists of a sequence of Q vectors of dimension R , and you want to train both an FTDNN and a Transformer as predictors.

- Describe the architecture of an FTDNN that you could train as a predictor. What will be the dimensions of the various weight matrices as well as the inputs and layer outputs of the network. Identify what parameters of the network (e.g., numbers of neurons) can be chosen, and how would you choose them.
- Describe the training process for the FTDNN. How would you format the data (inputs and targets) for training, what loss function would you use, etc.
- Describe the architecture of a transformer that you could train as a predictor. What will be the dimensions of the various weight matrices as well as the inputs and layer outputs of the network. Identify what parameters of the network can be chosen, and how would you choose them.

Seq2Seq and Attention

- iv. Describe the training process for the transformer. How would you format the data (inputs and targets) for training, what loss function would you use, etc.
- v. Compare and contrast the FTDNN and the transformer for the prediction task. What are the advantages and disadvantages of each? How could each be used for multi-step prediction?